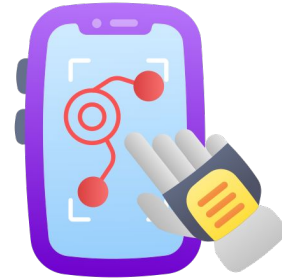


Real-time Control and Decision-making with Edge

MODULE 3.1





Agenda

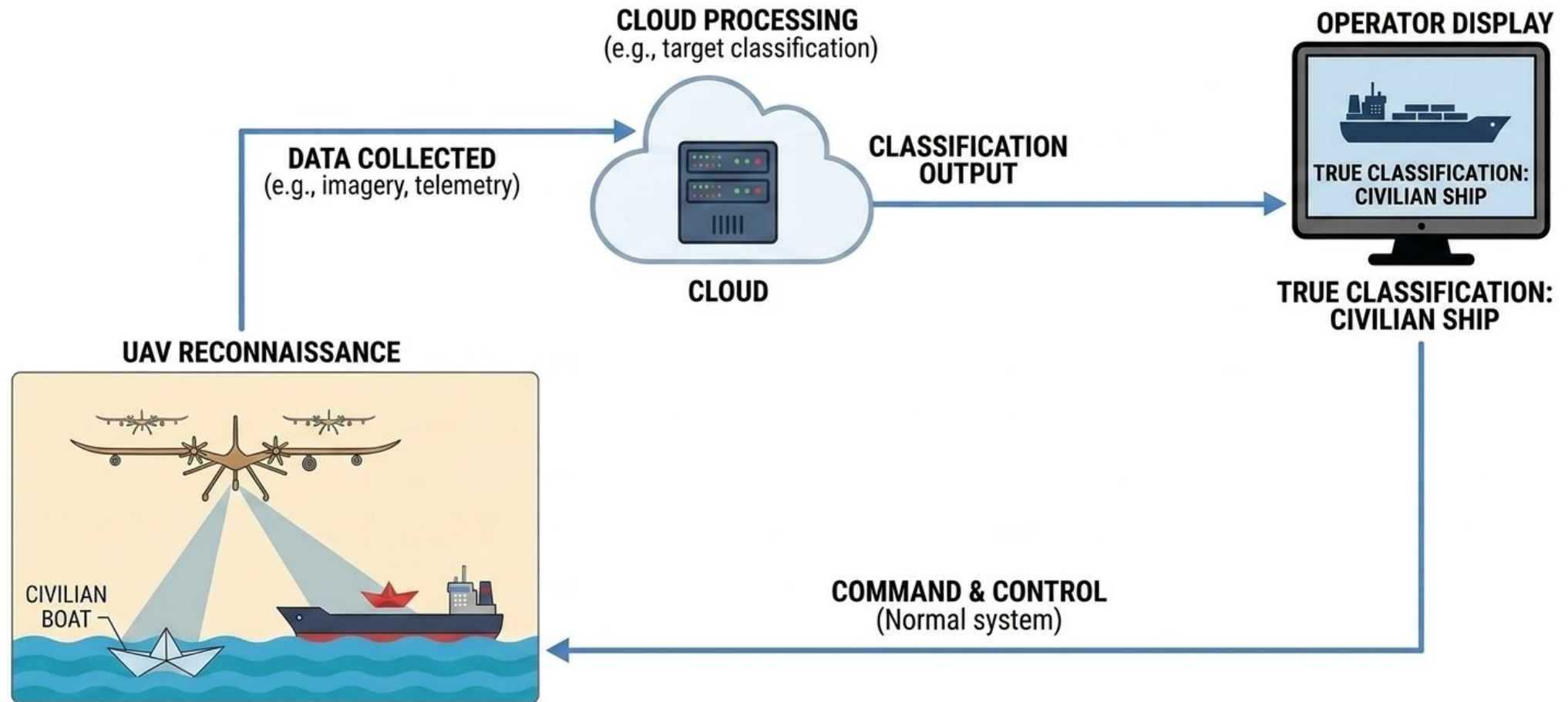
- Motivation: Systems Under Constraints
- The Paradigm Shift to Edge Computing
- Defining Real-Time Control
- The Environmental Constraints
- Intelligent Decision-Making & Offloading
- Transitioning to Case Studies
- Notebook 01: Slides 9-16
- Notebook 02: Slides 17-24
- Notebook 03: Slides 25-32
- Notebook 04:
- Concluding Remarks

Motivation: Systems Under Constraints

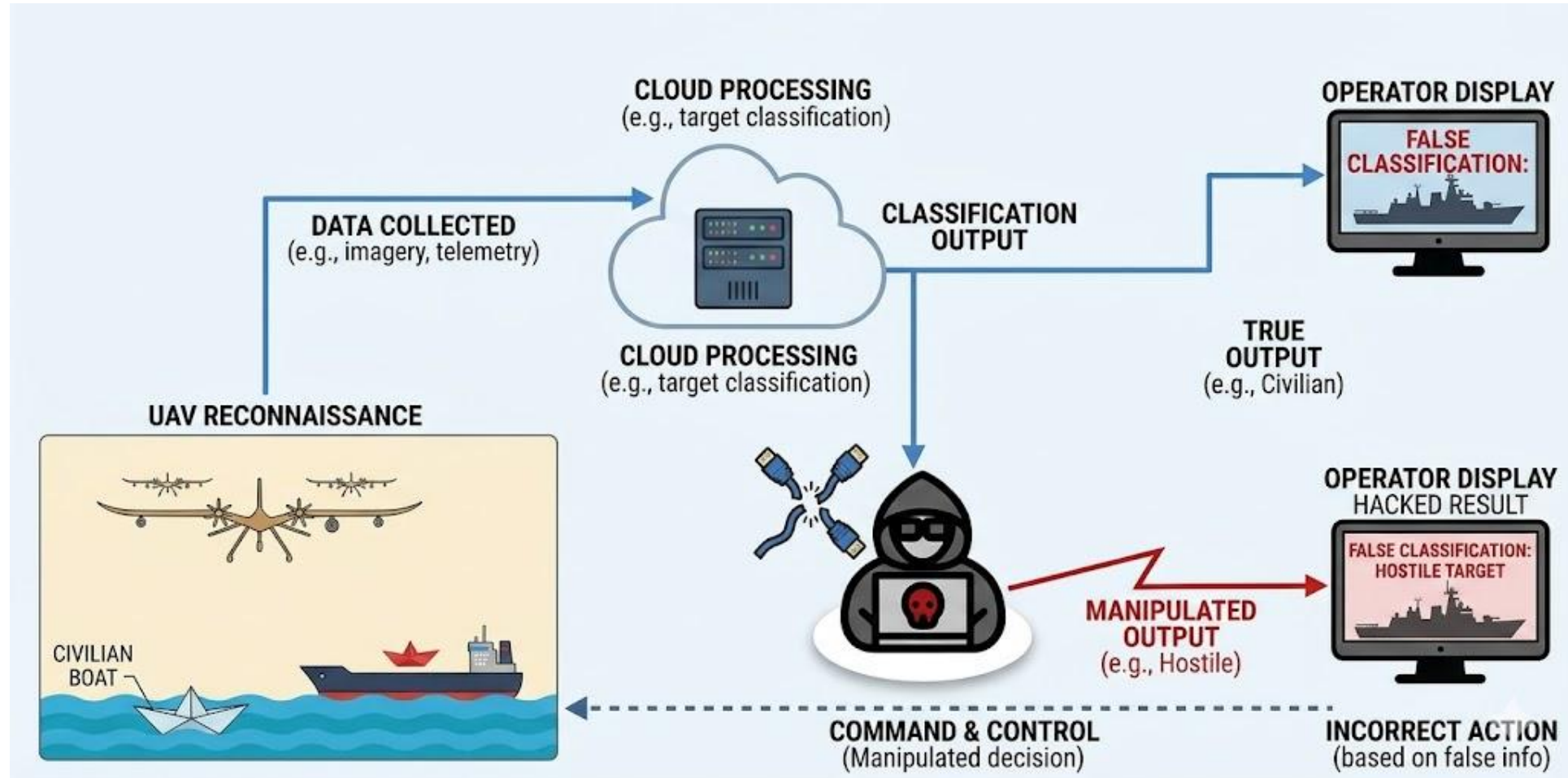
- Hardware latency in robotics can destabilize control loops.
- Real-world sensor noise creates gaps between simulation and reality.
- GenAI traffic bursts often lead to severe GPU congestion.
- We need adaptive hybrid systems to handle these variables.



Example

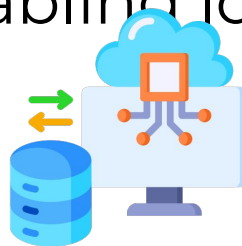


Example



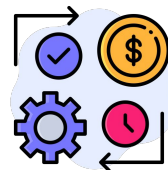
The Paradigm Shift to Edge Computing

- Cloud computing introduces unavoidable network latency and bandwidth bottlenecks.
- Edge computing pushes processing directly to the data source (sensors, robots, local gateways).
- Why it matters: Crucial for applications requiring sub-millisecond reaction times.
- Reduces reliance on continuous network connectivity, enabling localized autonomy.



Defining Real-Time Control

- Systems that continuously monitor and adjust dynamic environments (e.g., autonomous driving).
- Relies on a strict, continuous "Sense → Compute → Act" loop.
- The Danger of Latency: Delays in this loop can cause severe system instability or hardware failure.
- Traditional physics-based controllers (like PID) are now being augmented by AI for predictive adjustments.



The Environmental Constraints



Compute Limits: Edge devices (e.g., Raspberry Pi, onboard compute) have strict memory and thermal caps.



Sensor Reality: Real-world sensors (LiDAR, telemetry) produce highly noisy, high-variance data compared to simulations.



Network Jitter: Fluctuating network speeds make cloud-reliant control highly unpredictable.



The Solution: AI models must be aggressively optimized (via feature abstraction or quantization) to survive on the edge.



Intelligent Decision-Making & Offloading

- Edge compute is limited; not every complex task can or should be processed locally.
- Systems must dynamically answer: Compute locally or offload to the cloud?
- This decision requires real-time awareness of local GPU load, queue sizes, and network congestion.
- The ultimate goal is balancing strict Service Level Agreements (SLAs) against cloud egress costs.

Transitioning to the Case Studies



How do we solve these foundational challenges in practice?



Case 1 (Real-Time Control): Appliances Energy Prediction: Edge AI & Time-Series Forecasting



Case 2 (Edge Robotics): Compressing raw sensor perception into lightweight, real-time motor commands.

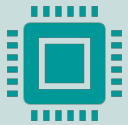


Case 3 (Decision-Making): Using neural networks to dynamically route GenAI tasks across a cluster.

NB:01 - Objective: Edge-Style Forecasting



Goal: Build a time-series forecasting pipeline using Ridge Regression, 1D-CNN, and GRU, plus an edge decision agent.

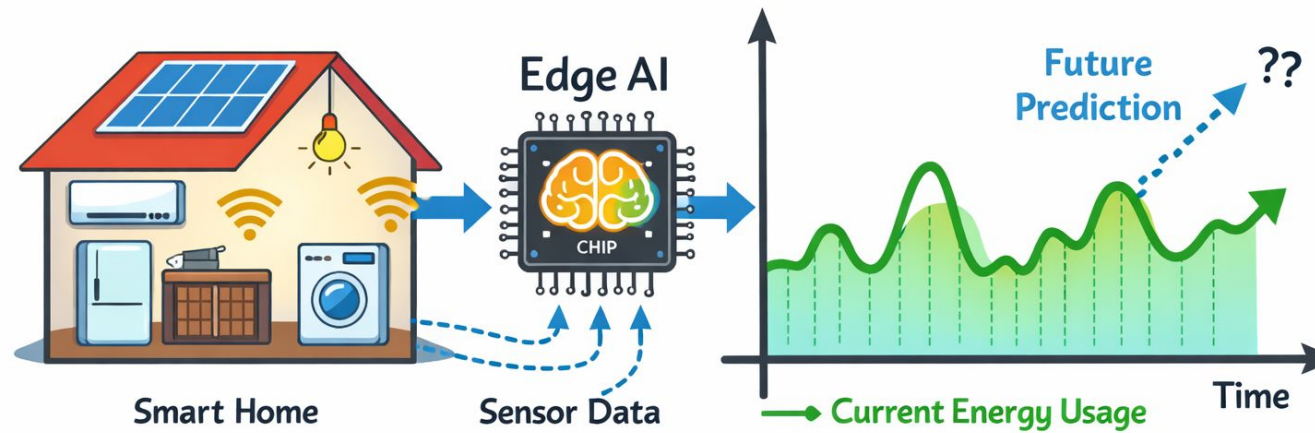


Application: Predict household appliance energy consumption and simulate real-time load shedding via an Edge Controller.



Constraint: Manage dynamic time-series patterns using windowed preprocessing and ensure reliable edge predictions.

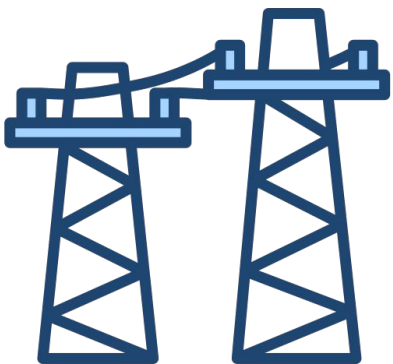
Schematic Diagram



Case Study 01: Schematic Diagram

NB:01 - Background

- Grid Instability: Residential peak loads are highly volatile and non-linear, threatening local grid stability.
- The Latency Gap: Traditional cloud processing is too slow for the sub-second decisions required for real-time load shedding.
- Edge Intelligence: An autonomous Edge Controller processes windowed data locally to simulate immediate, reliable load-shedding actions.



NB:01 - Data Preprocessing Strategy



Applied Robust Scaling to reduce impact of spikes and outliers in sensor readings.



Converted tabular telemetry into windowed time-series sequences for temporal learning.



Built a noise-resilient pipeline for stable predictions under volatile appliance usage.

NB:01 - Model Architectures Compared



Ridge Regression (Baseline): Linear benchmark on flattened windows for fast, interpretable predictions.



1D-CNN (Edge Model): Learns local temporal patterns in windowed sequences with efficient inference.

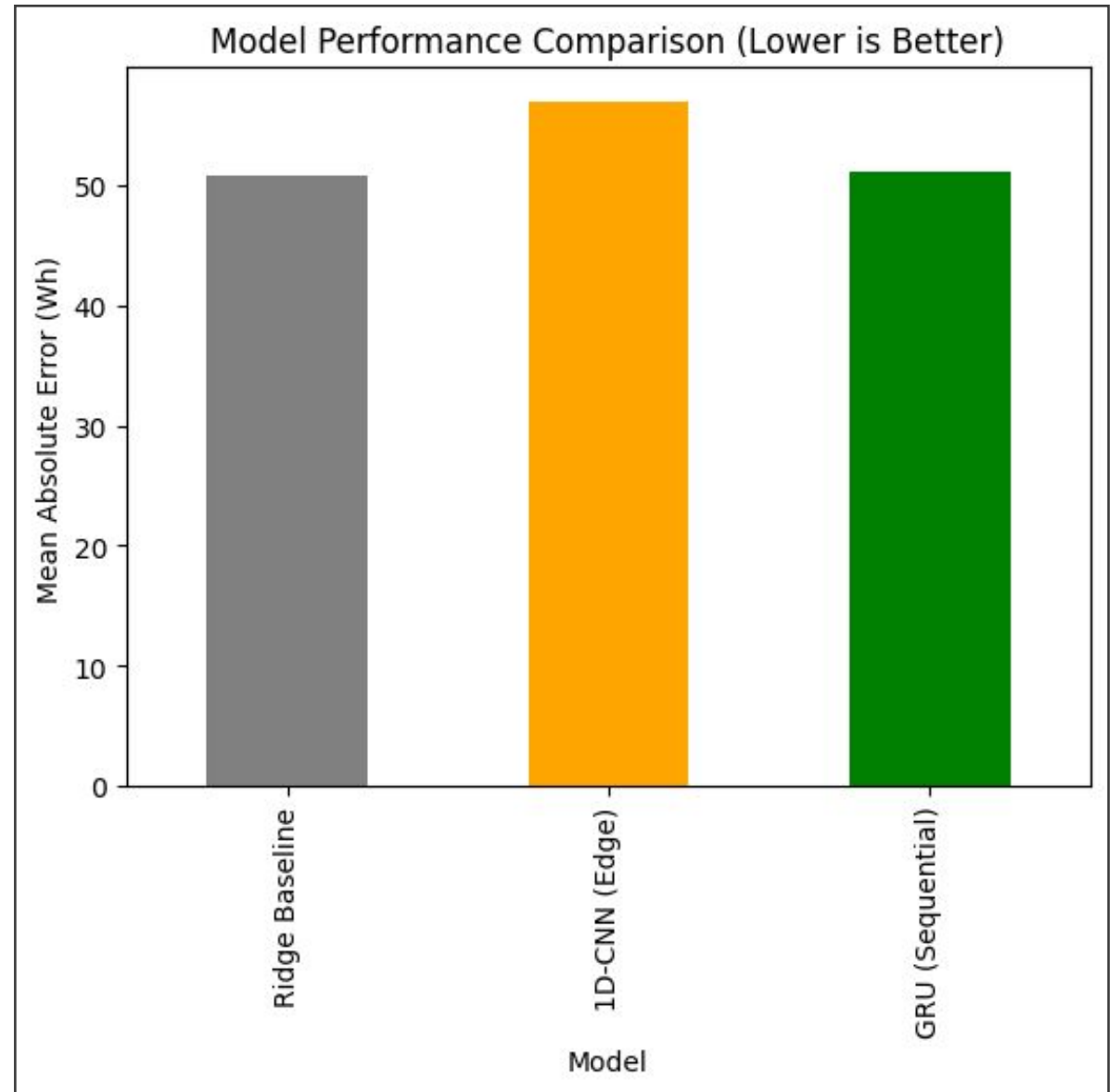


GRU (Sequential Model): Captures longer-term temporal dependencies in energy usage.

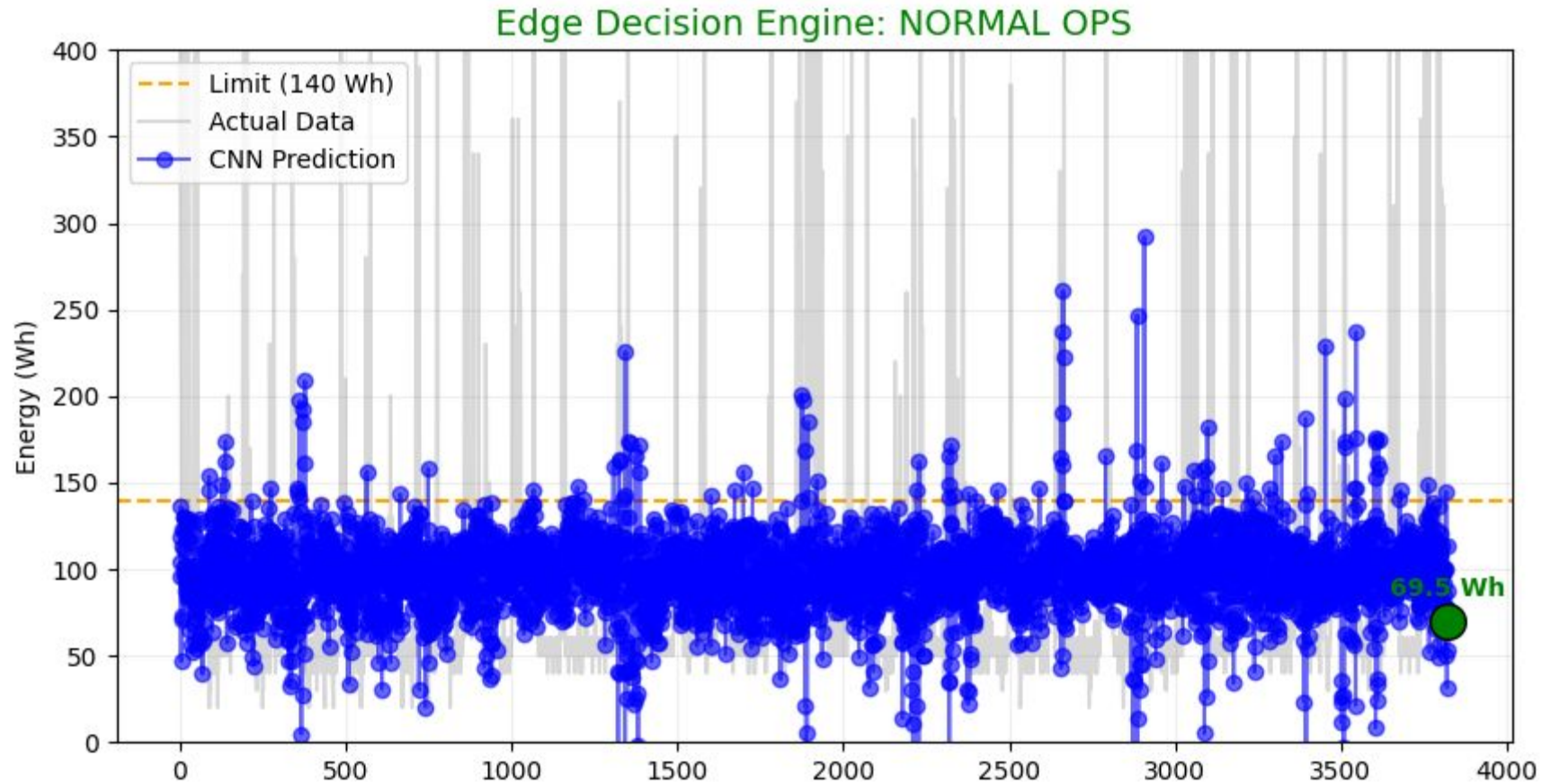


Conclusion:
GRU delivers the most accurate forecasts, outperforming both the linear baseline and CNN on sequential energy patterns.

NB:01 - Results and Edge Deployment



NB:01 - Results and Edge Deployment



NB:01 - Results and Edge Deployment

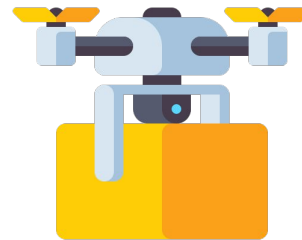
- Conversion of PyTorch models to TorchScript or ONNX.
- Target Hardware: Low-power devices like a Raspberry Pi.
- Future Work: Adding temporal features like 'hour-of-day' for peak usage.

NB01: Industrial Use case

- Smart Energy (Schneider Electric)
 - Use Case: Microgrid management for large retail centers.
 - The Problem: Expensive utility penalties during peak energy surges.
 - Solution: Time-series models predict peaks 15 minutes ahead.
 - Result: Auto-switches to battery power, saving millions in "peak charges."
 - <https://perspectives.se.com/blog-stream/ai-is-taking-energy-management-to-the-next-frontier>

NB:02 - Objective: Behavioral Cloning

- Goal: Train a robot to mimic an 'expert' agent's navigation tasks.
- Strategy: Map raw Perception (LiDAR) directly to Action (Motor Control).
- Advantage: Replaces brittle manual if-then rules with a fluid neural policy.



NB:02 - Background

- Heuristic Rigidity: Traditional "if-then" navigation rules often fail in complex, dynamic environments that the original programmer did not explicitly define.
- The Perception-Action Gap: Translating high-dimensional sensor data (like LiDAR) into precise motor commands manually is computationally expensive and error-prone.
- Dynamic Obstacle Complexity: Static algorithms struggle to maintain fluid trajectories when faced with unpredictable spatial layouts and moving objects.

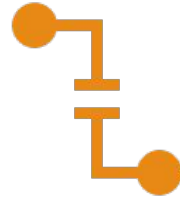
NB:02 - Sensor Abstraction (LiDAR)

- Raw LiDAR provides 360+ points of high-dimensional, noisy depth data.
- Technical Fix: 'Binning' points into spatial cones: Front, Left, and Right.
- Benefit: Minimizes noise and reduces computational overhead for the edge.

NB:02 - Robust Data Pipeline



Normalized LiDAR distances (clipped at 5m) to prevent gradient explosion.

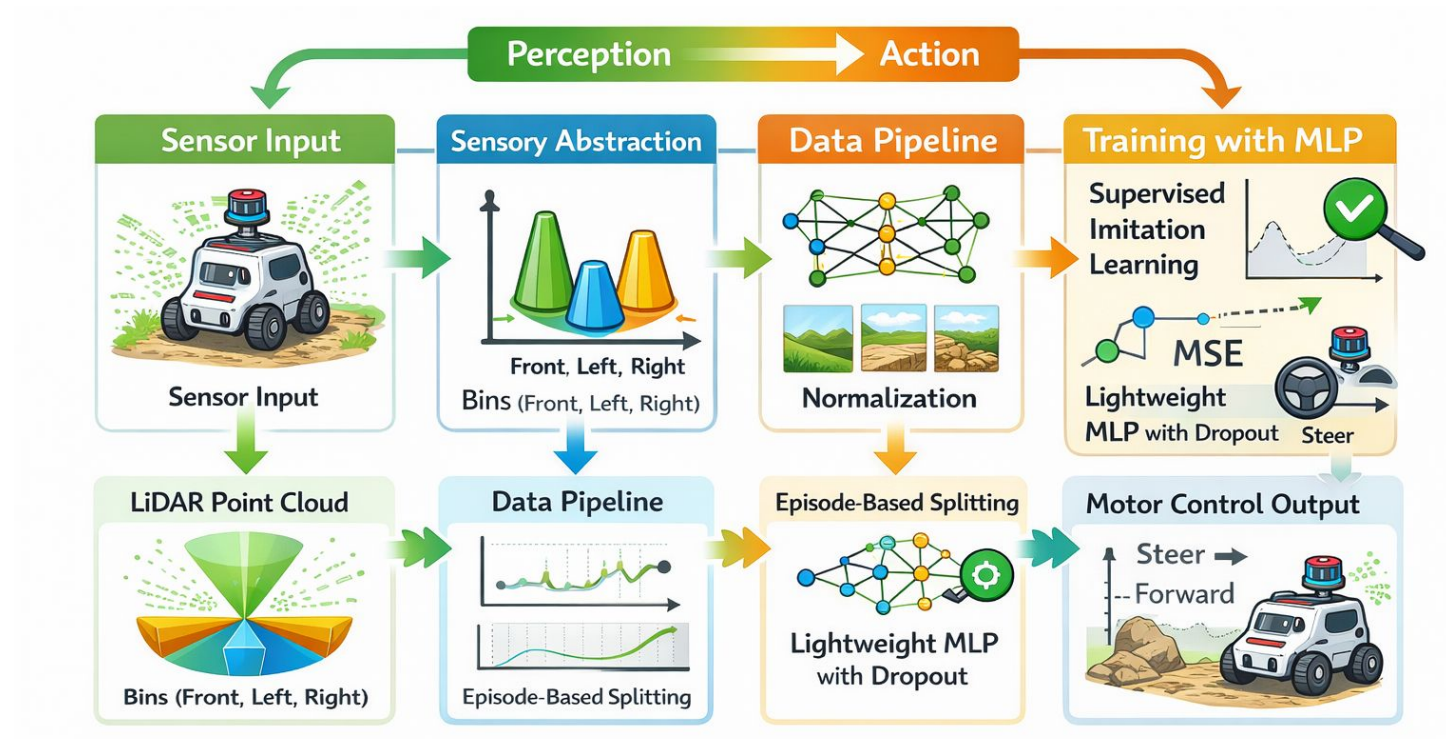


Data Splitting: Split by episodes (files) rather than randomized steps.



Why it matters: Prevents the model from memorizing specific room layouts.

Schematic Diagram



Case Study 02: Schematic Diagram

NB:02 - Baseline MLP Policy

- Architecture: Lightweight 3-layer MLP optimized for low-latency embedded execution.
- Robustness: Integrated Dropout (0.2) to mitigate stochastic sensor noise and failures.
- Objective: Maps 3D inputs to 2D motor commands via Mean Squared Error (MSE) loss.

NB:02 - Residual Learning Policy

- **Architecture:** Deep feature extractor utilizing **Skip Connections** to prevent signal degradation.
- **Stability:** Employs **Batch Normalization** to handle varied sensor dynamic ranges.
- **Performance:** Projects input to a **64-feature manifold** for complex obstacle negotiation.

NB:02 - Validation Outcomes

- Evaluated overall performance using RMSE analysis of predicted actions.
- Behavioral Visualization: Verified obstacles result in correct maneuvers.
- Confirmed driving consistency across smooth floors and rough surfaces.

NB02: Industrial Use case

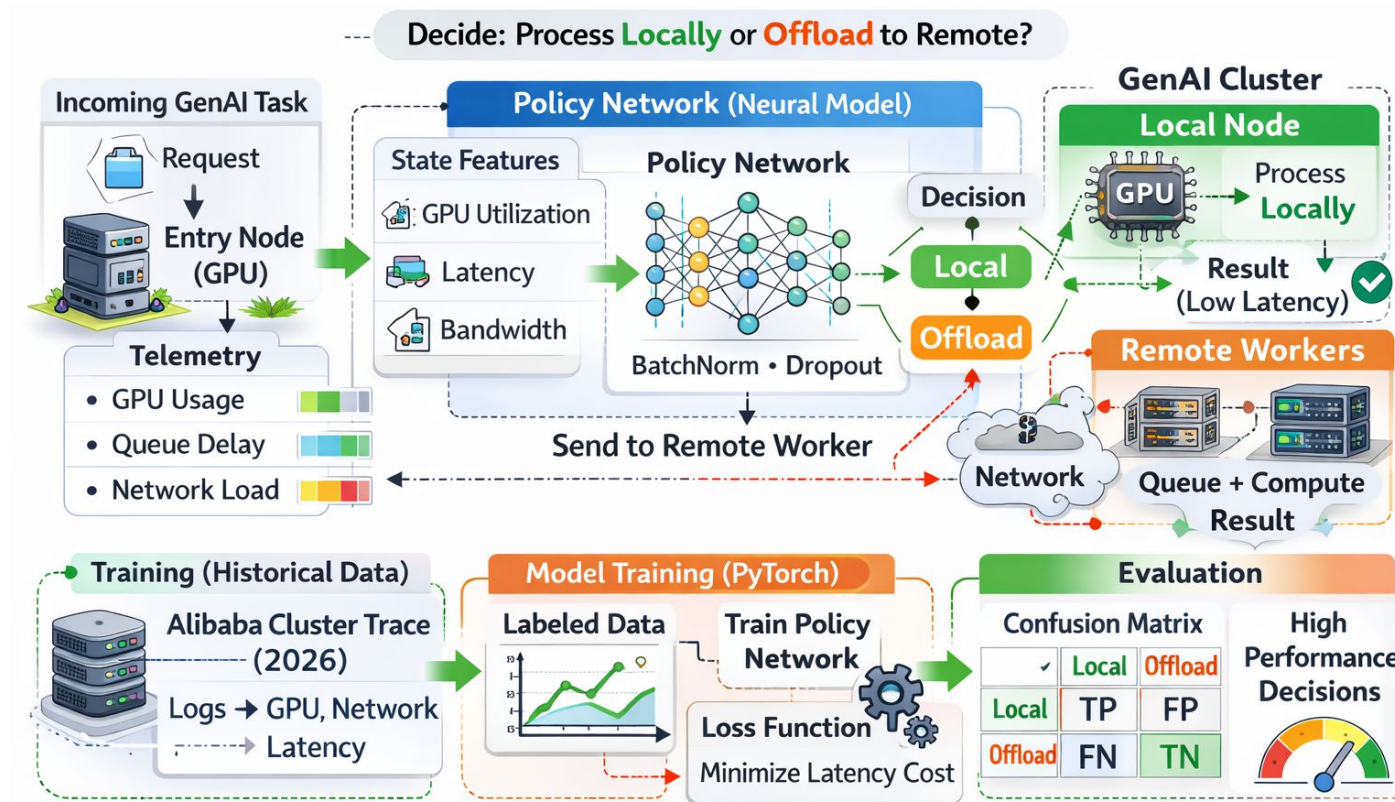
- Robotics & Logistics (Amazon Robotics)
 - Use Case: Autonomous navigation for "Proteus" warehouse robots.
 - The Problem: Fixed-path robots get stuck in dynamic, human-filled spaces.
 - Solution: Imitation learning mimics human navigation and safety logic.
 - Result: Robots safely navigate crowded floors without manual resets.



NB:03 - Objective: Neural Policy Network

- Goal: Decide if an inference task should be processed Locally or Offloaded.
- Decision Factors: Real-time network congestion and local GPU availability.
- Focus: Managing bursty GenAI workloads in hybrid environments.

NB:03 – Schematic Diagram



Case Study 03: Schematic Diagram

NB:03 - Background

Resource Constraints: Mobile and edge devices often lack the memory and compute power required to handle large-scale Generative AI (GenAI) workloads locally.

A light blue downward-pointing arrow indicating a flow from the first point to the second.

Bursty Workloads: AI inference requests are often unpredictable and non-uniform, leading to sudden "bursts" that can overwhelm local hardware.

A light blue downward-pointing arrow indicating a flow from the second point to the third.

Network Volatility: Relying entirely on the cloud introduces latency risks due to real-time network congestion and varying bandwidth.

NB:03 - Real-World Telemetry

Utilized authentic production logs (Alibaba Cluster Trace 2026).

Key Inputs: Inference Latency, Queue Response Time, and GPU Duty Cycle.

Allowed the model to learn from hardware behavior and network jitter.

NB:03 - Solving Class Imbalance

- Pitfall: Infrastructure data is often biased toward one decision (e.g., local).
- Fix: Implemented Median-Based Cost Scoring for a balanced boundary.
- Logic: Label 1 if cost > median (Congested), Label 0 if cost < median (Efficient).

NB:03 - Offloading Strategy Model

- Architecture: Hybrid PyTorch network utilizing BatchNorm and Dropout for stability.
- Goal: Predicts the optimal execution venue (Local vs. Cloud) to minimize total latency.
- Adaptability: Engineered for real-time response to dynamic network traffic spikes.



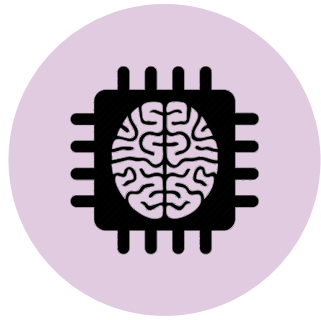
NB:03 - Neural vs. Classical Baselines

- ML Baselines: Compared against Logistic Regression and Random Forest (100 trees).
- Neural Baseline: Benchmarked with a Simple MLP (16-unit hidden layer) to justify complexity.
- Evaluation: Models validated using Cross-Entropy Loss and Confusion Matrices for offloading accuracy.

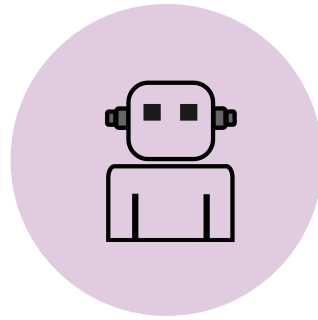
NB03: Industrial Use case

- Cluster Intelligence (Verizon & AWS)
 - Use Case: 5G-enabled smart traffic monitoring.
 - The Problem: Cloud latency is too slow for real-time accident alerts.
 - Solution: Offloads video processing to the nearest 5G cell tower (Edge node).
 - Result: Latency dropped from 100ms to 10ms, enabling instant emergency response.

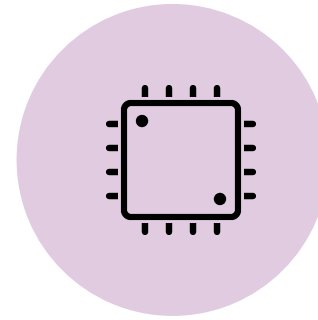
Unified Technical Theme



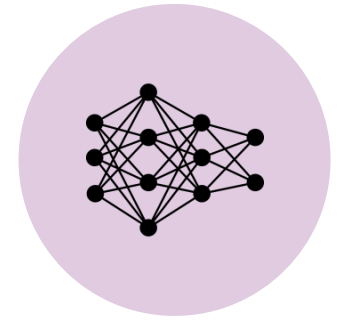
CENTRAL THEME: INTELLIGENT
DECISION-MAKING UNDER
HARDWARE CONSTRAINTS.



ROBOTICS (NB:02): FOCUSED
ON LOW-LATENCY EDGE
CONTROL.



DISTRIBUTED SYSTEMS (NB:03):
FOCUSED ON THROUGHPUT
OPTIMIZATION AND SLAS.



Medical RAG (NB:04): FOCUSED
ON BUILDING LIGHTWEIGHT LLM
SOLUTION FOR SENSITIVE
DATA.

Key Performance Lessons

Feature abstraction (LiDAR binning) is non-negotiable for the edge.

Data-driven neural policies consistently outperform static schedulers.

Robust preprocessing is essential when dealing with noisy physical sensors.

Use smaller LLM when dealing with local sensitive data.

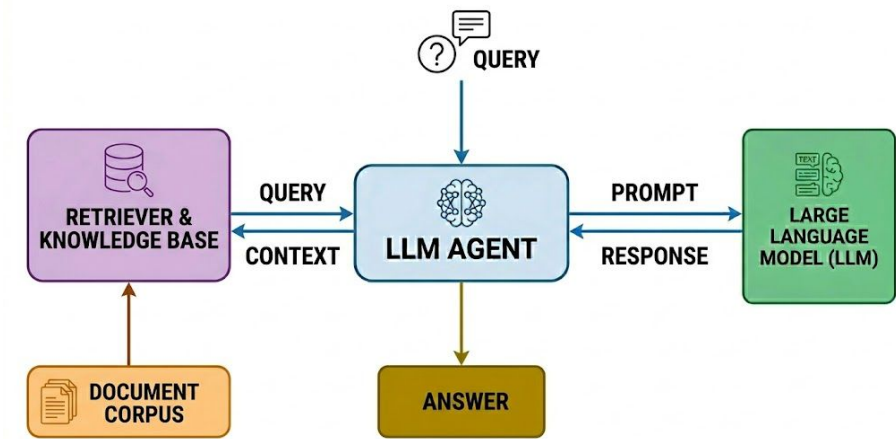
LLM Agents

- LLM agents are independent systems capable of taking actions
- It consists of four components: Agent code (LLM), memory, tool and planning
- Extensively used for multiple tasks like question answering, bug fixing and labeling.



RAG Agent

- RAG combines a document retriever with an LLM to generate answers grounded in specific external data
- It provides access to information outside the LLM's training set.
- Can be used for secure, local processing of sensitive clinical records



NB:04 - RAG System For Medical QA

- Uploads user's medical documents in the system
- It reads the documents, splits into chunks and converts to vectors
- When the user asks questions, it retrieves similar segments and answers from it

Technical Comparison of Case Studies

Feature	NB:01 (Energy Forecasting)	NB:02 (Imitation Learning)	NB:03 (Cluster Offloading)	NB:04 (RAG Chatbot)
Primary Goal	Time-series prediction	Sensor-to-Action mapping	Resource Scheduling	Small Model
Input Data	Power Telemetry (Amps/Volts)	Binned LiDAR (3 Cones)	Cluster Logs (GPU/Network)	User's document
Model Type	Sequential (GRU / CNN)	Lightweight MLP	Neural Policy Network	LLM
Edge Constraint	Power spikes & Outliers	Real-time motor latency	SLA & Traffic Bursts	Limited GPU and Data

Future Directions

- Deployment: Transitioning to edge hardware via TorchScript/ONNX.
- Feature Engineering: Enhancing forecasting with temporal metadata.
- Bridging the gap by training on noisy physical sensor data.

